

CSCE 647 IMAGE SYNTHESIS · MODULE 11 · LECTURE NOTES

Bidirectional Methods

Building paths from both ends, and when it is worth it.

MODULE OUTCOMES

- Explain why path tracing fails on caustics and SDS paths.
- Describe light tracing and its complementary strengths.
- Explain bidirectional path tracing and its connection strategies.
- Explain photon mapping and what kind of bias it has.
- Choose an algorithm from the transport a scene contains.

1 What path tracing cannot find

THE CAUSTIC PROBLEM

Consider light focused through a glass sphere onto a floor. The transport path is: eye → floor (diffuse) → glass (**specular**) → light.

Next-event estimation cannot help at the specular vertex. As Module 08 established, a shadow ray from a specular surface has zero probability of being the exact refracted direction, so the light sampling strategy contributes nothing there.

So the path must find the light by BSDF sampling: refract through the glass and happen to strike the light. For a small light and a narrow refracted cone, the probability is minute — and when it succeeds, the contribution is enormous, because the caustic is bright.

That is precisely the firefly condition of Module 05: rare events with large values. The caustic appears as scattered bright pixels that converge extraordinarily slowly, and it is the standard demonstration that unidirectional path tracing is incomplete rather than merely slow.

1.1 Heckbert path notation

Notation	Meaning	Difficulty
L, E	Light source; eye.	—
D, S	A diffuse vertex; a specular vertex.	—
LDE	Light, one diffuse bounce, eye. Direct lighting.	Easy. NEE handles it.

Notation	Meaning	Difficulty
LD*E	Any number of diffuse bounces.	Path tracing's home ground. This is what it is good at.
LSDE	Light, specular, diffuse, eye. A caustic.	Hard for path tracing, easy for light tracing and photon mapping.
LSDSE	Specular–diffuse–specular. A light under water, viewed through the glass of the tank.	Hard for everything. See below.

The notation is worth learning: the literature assumes it, and it names a transport problem precisely enough to reason about which algorithm can handle it.

SDS PATHS DEFEAT BOTH DIRECTIONS

A specular–diffuse–specular path requires reaching a diffuse surface through a specular chain from *both* ends. The canonical examples are a light submerged in a water tank viewed through the glass, and a car headlight behind its lens reflected in a wet road.

From the eye: you must traverse the specular chain and then hit the light. Probability near zero.

From the light: you must traverse the other specular chain and then hit the eye. Also near zero.

And bidirectional path tracing does not rescue it, because the connection step between the camera and light subpaths must land on a vertex where a deterministic connection has non-zero contribution — and a specular vertex does not. Every BDPT strategy for an SDS path has zero or near-zero probability.

This is the transport that Metropolis light transport and photon mapping exist to handle, and it is why those methods survive despite their disadvantages.

2 Light tracing

	Path tracing	Light tracing
Starts at	The camera.	A light source.
Connects to	Lights, by shadow ray (NEE).	The camera, by projecting the vertex onto the sensor.
Good at	Directly visible surfaces; diffuse interreflection; most ordinary transport.	Caustics; light through glass; illumination that is hard to find from the eye.
Bad at	Caustics; small bright lights behind specular surfaces.	Directly visible surfaces — trivially easy from the eye, and wasteful to find from the light.
Pixel assignment	Every path contributes to one known pixel.	Paths splat into whichever pixel they happen to reach, so the image must be accumulated rather than computed per pixel.

LIGHT TRACING NEEDS A CAMERA WITH AN APERTURE

To connect a light path to the camera you must evaluate the camera's **importance function** — the sensitivity of a given pixel to radiance arriving from a given direction. This is the exact dual of a light's emitted radiance, and Veach's measurement equation is written to make the symmetry explicit.

A pinhole camera's importance is a delta distribution. The probability that a randomly generated light path passes exactly through an infinitesimal pinhole is zero, so no light path can ever be connected to it by chance.

So light tracing requires either a camera with a finite aperture — a thin-lens model — **or a deterministic connection step** that projects the vertex onto the sensor plane and evaluates the importance there.

The symmetry with Module 09's delta lights is exact. A point light cannot be hit by a BSDF-sampled ray and exists only through explicit connection; a pinhole camera cannot be hit by a light path and exists only through explicit connection. The same mathematics, mirrored.

3 Bidirectional path tracing

- 1 Trace a subpath from the camera, producing vertices $z_0, z_1, z_2, \dots$
- 2 Trace a subpath from a light, producing $y_0, y_1, y_2, \dots$
- 3 **Connect every camera vertex to every light vertex** with a shadow ray, evaluating the BSDFs at both ends and the geometry term between.
- 4 A complete path with k vertices can be constructed in $k+1$ different ways, depending on how many vertices came from each subpath. **These are different sampling strategies for the same path.**
- 5 **Combine them with multiple importance sampling.**

Step 5 is the reason BDPT works, and it is why MIS and BDPT appear in the same thesis: Veach needed MIS in order to make BDPT function. Without a principled way to weight the many strategies that can generate the same path, the method either double-counts or must arbitrarily discard strategies.

Note that the family of strategies *includes* the ones we already know. Connecting the camera subpath directly to a light is path tracing with NEE; connecting a light subpath directly to the camera is light tracing. BDPT generalises both and adds every intermediate option.

STRICTLY MORE CAPABLE, FREQUENTLY SLOWER

BDPT handles everything path tracing handles, plus caustics and difficult indirect illumination — light entering a room through a small gap, for instance — without any per-scene configuration.

But each sample costs far more. Connecting every pair of vertices is $O(k^2)$ shadow rays per path pair, and on an ordinary diffuse scene the great majority of those extra strategies contribute almost nothing to the final image.

At equal time it frequently loses to a well-implemented path tracer with MIS on the kinds of scene people actually render. It wins decisively on the scenes it was designed for and ties or loses elsewhere.

It is also considerably harder to implement correctly. The MIS weights must account for the probability of generating each path under every strategy, including ones that were not used, and a subtle error produces a result that is wrong only in certain configurations. Production renderers typically offer BDPT and default to path tracing.

4 Photon mapping

A two-pass method that decouples finding light transport from shading.

- 1 **Pass 1:** trace photons from the light sources into the scene, storing each interaction — position, incoming direction, and power — in a spatial data structure, typically a kd-tree or a hash grid.
- 2 **Pass 2:** trace paths from the camera. At each vertex where a radiance estimate is needed, gather the nearest photons within some radius and estimate the incoming radiance from their density and power.

Property	Assessment
Biased	The gather averages photons over a finite area, so the estimate is not local. Radiance is blurred spatially — which is why photon-mapped caustics look slightly soft even when fully converged.
Consistent	As the photon count grows the gather radius can shrink, and the bias vanishes in the limit. This is the standard example of the consistent-but-biased category from Module 05.
Excellent at caustics	Photons travel from the light <i>through</i> the glass and land exactly where the caustic is bright. The density of stored photons is naturally proportional to the illumination, which is precisely what the hard case needs.
Memory-bound	Quality is limited by how many photons fit in memory. Progressive photon mapping removes this constraint by running many passes with a shrinking radius and accumulating, trading memory for time and remaining consistent.

5 Metropolis light transport

EXPLORE THE NEIGHBOURHOOD OF PATHS THAT WORKED

MLT treats the space of all light paths as a distribution to be sampled and applies Markov chain Monte Carlo: having found a path that carries significant energy, propose a small mutation of it, and accept or reject according to the Metropolis rule.

Once a difficult path has been found, its neighbours are found almost for free. That is exactly the property SDS transport requires — such paths are extremely rare but strongly clustered in path space, so a method that explores locally once it arrives is ideally suited.

The costs are substantial. Successive samples are correlated, so the error does not look like noise — it looks like splotches and uneven brightness, which many people find more objectionable. Convergence is difficult to assess, because the usual variance estimates assume independence. And start-up bias, mutation design, and chain management are all genuinely hard to get right.

In production it is used for specific difficult shots rather than as a default. Primary-sample-space MLT, which mutates the random numbers driving an ordinary path tracer rather than the path vertices themselves, is far easier to implement and is the version to try first.

6 Choosing

Scene characteristics	Best choice	Why
Diffuse interior lit by area lights.	Path tracing with MIS.	LD*E transport is exactly what it is good at.
Caustics through glass or water.	Photon mapping, or BDPT.	Light-side construction finds LSDE paths naturally.
Strong indirect through a small opening.	BDPT.	Light subpaths get through the opening; camera subpaths struggle to find it.

Scene characteristics	Best choice	Why
SDS paths — submerged lights, lens caustics.	MLT, or photon mapping.	Nothing else can construct these paths with non-zero probability.
General production work.	Path tracing + MIS + a denoiser.	What almost everyone ships.

WHY THE SIMPLE METHOD WON

Path tracing with MIS is simple to implement, robust across scene types, trivially parallel, and has flat, predictable memory use. Those properties matter enormously in a production pipeline, where an algorithm that is occasionally twice as fast and occasionally catastrophically wrong is worse than one that is uniformly adequate.

The sophisticated methods each win decisively on a narrow class of transport and tie or lose elsewhere, while costing far more to implement, tune, and maintain.

Then denoising changed the economics entirely. If a noisy path-traced image at 64 samples can be cleaned to an acceptable result, BDPT's variance advantage on a subset of scenes no longer justifies its complexity. Module 12 covers this.

Learn these methods anyway, and not out of completeness. They give you the vocabulary to say what kind of transport a scene contains — and when your default renderer refuses to converge on a shot, knowing that it is an SDS problem rather than a sampling problem is the difference between fixing it and not.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Veach thesis — Chapters 10 and 11, BDPT and MLT	The original source for both methods, including the measurement equation and the camera importance function of §2.
PBRT 4th ed. — Chapter 13 and the online BDPT chapter	BDPT with complete code, including the MIS weight computation, which is the part that is hard to get right.
Jensen — Realistic Image Synthesis Using Photon Mapping	Photon mapping from its author; the papers are free on this page.
Hachisuka & Jensen — Progressive Photon Mapping (free)	The method that removed photon mapping's memory limit while keeping consistency.
Kelemen et al. — Primary Sample Space MLT (free)	The far simpler MLT variant mentioned in §5. Start here if you implement one.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Render a glass sphere above a diffuse floor with your path tracer. Report how many samples are needed before the caustic is acceptable, and whether it ever is.
- 2 Classify ten transport paths in a scene of your choosing using Heckbert notation, and predict which your path tracer will struggle with.
- 3 Implement light tracing. Render the glass sphere scene with it and compare against path tracing at equal time.
- 4 Demonstrate the pinhole problem: attempt light tracing with a pinhole camera and explain the result. Then add a finite aperture.
- 5 Construct an SDS scene — a light inside a glass enclosure, viewed through another glass surface. Render it with path tracing and with light tracing and document that both fail.
- 6 Implement BDPT with the full set of connection strategies. Verify that disabling all strategies except one reproduces path tracing, and another reproduces light tracing.
- 7 For a BDPT implementation, render each connection strategy to a separate image buffer. Looking at which strategies contribute where is the clearest possible explanation of the algorithm.
- 8 Implement basic photon mapping for caustics only, combined with path tracing for everything else. Compare the caustic quality against path tracing at equal time.
- 9 Vary the photon gather radius and show the bias explicitly: the caustic should visibly blur as the radius grows.
- 10 Write a page recommending an algorithm for three scenes of your choice, justifying each from the transport they contain.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does path tracing fail on caustics? Be precise about which strategy fails and why.
- 2 Write Heckbert notation for direct lighting, diffuse GI, a caustic, and the hard case.
- 3 Why do SDS paths defeat both path tracing and light tracing, and why does BDPT not fix them?
- 4 What is light tracing good at, and why does it need a camera with an aperture?
- 5 What is the camera importance function, and what is it the dual of?
- 6 Describe BDPT's connection strategies and say why MIS is essential to it.
- 7 Why is BDPT often slower than path tracing at equal time despite being more capable?
- 8 In what sense is photon mapping biased, and in what sense consistent?

- 9 What property of SDS transport makes Metropolis suited to it, and what does MLT's error look like?
- 10 Why did path tracing with MIS win, and what did denoising change?